Digital Logic Design

Dr. Diman Siddiq Hassan

Diman.Hassan@uoz.edu.krd

Binary System Complements

As the binary system has base r = 2. So, the two types of complements for the binary system are **1's complement and 2's complement**.

- **1's complement**

The 1's complement of a number is found by changing all 1's to 0's and all 0's to 1's. This is called as taking complement or 1's complement. Example of 1's Complement is as follows:

- The 1’s complement of the following binary number 00001100 is 11110011.

- **2’s Complement Representation** The 2’s complement is formed in a way similar to forming the one’s complement: complement all of the bits in the number, but then add 1, and if that addition results in a carry-out from the most significant bit of the number, discard the carry-out.

2’s complement = 1’s complement + 1

For example: the 2’s complement of the binary number (10101) is 01010 + 1 = 01011

- The (*r* - 1)'s complement of octal or hexadecimal numbers is obtained by subtracting each digit from 7 or F (decimal 15), respectively.

- Examples in decimal base 10 complements:

1. The 9's complement of 546700 is 999999 - 546700 = 453299.
2. The 9's complement of 012398 is 999999 - 012398 = 987601.

- Examples in binary base 2 complements

1. The 1's complement of 1011000 is 0100111.
2. The 1's complement of 0101101 is 1010010.
3. **The 2's complement of the binary number (**0101101**) is**

1010010 + 1 = 1010011

- It is also worth mentioning that **the complement of the complement restores the number to its original value .**
-
- To see this relationship, note that the *r*'s complement of *N* is r^n - *N*, so that the complement of the complement is

r^n - (r^n - *N*) = *N* and is equal to the original number.

Arithmetic Operation

Subtraction with Complements

- The direct method of subtraction taught in elementary schools uses the borrow concept. In this method, we borrow a 1 from a higher significant position when the minuend digit is smaller than the subtrahend digit. The method works well when people perform subtraction with paper and pencil.

- However, when subtraction is implemented with digital hardware, the method is less efficient than the method that uses complements.

- The subtraction of two *n*-digit unsigned numbers *M* - *N* in base *r* can be done as follows:
- Example: Subtract *M* – *N* = 72532 – 3250.
- Hint: Both numbers should be in the same number of digits when subtracting.

1. Add the minuend *M* to the *r*'s complement of the subtrahend *N*. Mathematically, $M + (r^n - N) = M - N + r^n$.

2. If $M \geq N$, the sum will produce an end carry r^n, which can be discarded; what is left is the result *M* - *N*.

3. If $M < N$, the sum does not produce an end carry and is equal to $r^n - (N - M)$, which is the *r*'s complement of (*N* - *M*). To obtain the answer in a familiar form, take the *r*'s complement of the sum and place a negative sign in front.

Example of a 10'th complement subtraction

- Using 10's complement, subtract 72532 - 03250. (M >= N)

M = 72532

- 10's complement of N = + 96750 (9'th complement + 1)

Sum = 169282

- Discard end carry 10^5 = - 100000

Answer = 69282

==========

- Using 10's complement, subtract 3250 - 72532. (M < N)

M = 03250

10's complement of N = + 27468

Sum = 30718

- There is no end carry. Therefore, the answer is:
- - (10's complement of 30718) = -69282.

Example of binary subtraction using complements

- Given the two binary numbers *X* = 1010100 and *Y* = 1000011, perform the subtraction **(a)** *X* - *Y* and **(b)** *Y* - *X* by using 2's complements.

(a)

X =	1010100	(84)
2's complement of *Y* =	0111101 +	(67)
Sum =	10010001	
Discard end carry 2^7 =	- 10000000	
Answer of X - Y =	0010001	(17)

(b) (67 -84) = -(17)

Y = 1000011
2's complement of X = 0101100 +
Sum = 1101111

There is no end carry. Therefore, the answer is:

$Y - X$ = -(2's complement of 1101111) = - 0010001.

Repeat the previous example , but this time using 1's complement.

(a) $X - Y = 1010100 - 1000011 = (84 - 67)_{10}$

X = 1010100

1's complement of Y = 0111100 +

Sum = 10010000

End-around carry = + 1

Answer: $X - Y = 0010001 = (17)_{10}$

(b) $Y - X = 1000011 - 1010100$

Y = 1000011

1's complement of X = 0101011 +

Sum = 1101110

There is no end carry. Therefore, the answer is:

$Y - X$ = -(1's complement of 1101110) = (- 0010001)

Signed Numbers

- Positive integers (including zero) can be represented as unsigned numbers.
- In ordinary arithmetic, a negative number is indicated by a minus sign and a positive number by a plus sign.
- Because in computers everything must be represented with binary digits. A sign with a bit placed in the leftmost position of the number must be used.
- The convention is to make the sign bit 0 for positive and 1 for negative.
- The user determines whether the number is signed or unsigned.
- If the binary number is signed, then the leftmost bit represents the sign and the rest of the bits represent the number (sign-magnitude).
- If the binary number is assumed to be unsigned, then the leftmost bit is the most significant bit (MSB) of the number

An example

- Consider the number 9 = 00001001, represented in binary with eight bits.
- +9 is represented with a sign bit of 0 in the leftmost position, followed by the binary equivalent of 9, which gives 00001001.
- Although there is **only one way to represent +9**, there are **three different ways to represent -9** with eight bits:

1. signed-magnitude representation: 10001001
2. signed-1's-complement representation: 11110110
3. signed-2's-complement representation: 11110111

All the three types obtained from the original positive +9.

Arithmetic Addition

The addition of two signed binary numbers with negative numbers represented in signed-2's-complement form is obtained from the addition of the two numbers, including their sign bits. A carry-out of the sign-bit position is discarded.

+ 6	00000110	- 6	11111010	(2's comp. of 6)
+13	00001101	+13	00001101	
+19	00010011	+ 7	00000111	

+ 6	00000110	- 6	11111010 (2's comp. of 6)
-13	11110011	-13	11110011 (2's comp. of 13)
- 7	11111001	-19	11101101

- Note that negative numbers must be initially in 2's-complement form and that if the sum obtained after the addition is negative, it is in 2's-complement form. For example, -7 is represented as 11111001, which is the 2's complement of +7.

- In each of the four cases in the previous slide , the operation performed is addition with the sign bit included. Any carry out of the sign-bit position is discarded, and negative results are automatically in 2's-complement form.

Binary Codes

- Digital data is represented, stored and transmitted as group of binary bits. This group is also called as **binary code**.
- Digital systems represent and manipulate not only binary numbers, but also many other discrete elements of information.
- Any discrete element of information that is distinct among a group of quantities can be represented with a binary code (i.e., a pattern of 0's and 1's).
- The codes must be in binary because, in today's technology, only circuits that represent and manipulate patterns of 0's and 1's can be manufactured economically for use in computers.
- An *n*-bit binary code is a group of *n* bits that assumes up to 2^n distinct combinations of 1's and 0's, with each combination representing one element of the set that is being coded.

Binary Codes

- A set of four elements can be coded with two bits, with each element assigned one of the following bit combinations: 00, 01, 10, 11.
- A set of eight elements requires a three-bit code and a set of 16 elements requires a four-bit code.
- The bit combination of an *n*-bit code is determined from the count in binary from 0 to 2^n - 1.
- Each element must be assigned a unique binary bit combination, and no two elements can have the same value; otherwise, the code assignment will be ambiguous.

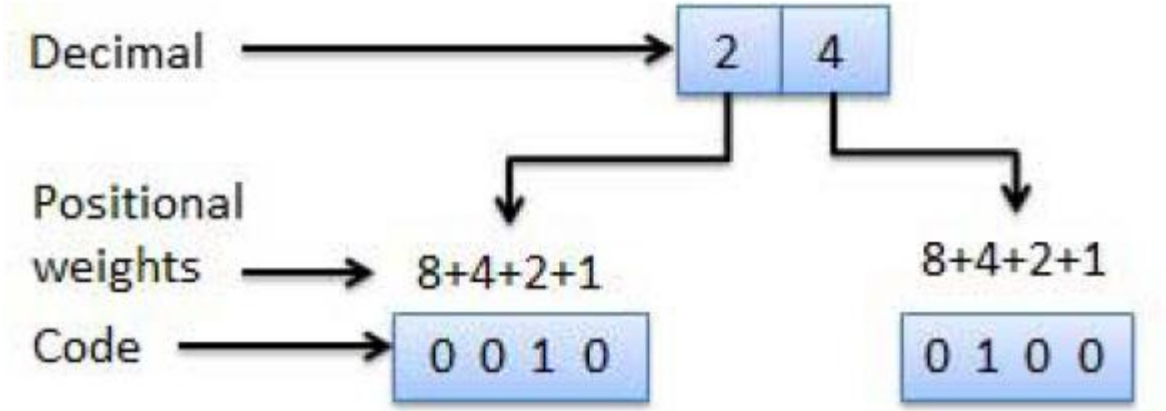
Classification of binary codes

The codes are broadly categorized into following categories.

1. Weighted Codes
2. Binary Coded Decimal Code (BCD)
3. Non-Weighted Codes
4. Alphanumeric Codes
5. Error Detecting Codes
6. Error Correcting Codes

Weighted Codes

- Weighted binary codes are those binary codes which obey the positional weight principle. Each position of the number represents a specific weight. Several systems of the codes are used to express the decimal digits 0 through 9. In these codes each decimal digit is represented by a group of four bits.



Binary-Coded Decimal (BCD) Code

- A binary code will have some unassigned bit combinations if the number of elements in the set is not a multiple power of 2.
- The 10 decimal digits form such a set. A binary code that distinguishes among 10 elements must contain at least four bits, but 6 out of the 16 possible combinations remain unassigned.
- A number with *k* decimal digits will require 4*k* bits in BCD.
- One decimal digit is represented by four bit code.
- Decimal 396 is represented in BCD with 12 bits as 0011 1001 0110, with **each group of 4 bits representing one decimal digit.**

- A decimal number in BCD is the same as its equivalent binary number only when the number is between 0 and 9. A BCD number greater than or equal to 10 looks different from its equivalent binary number, even though both contain 1's and 0's.
- Moreover, **the binary combinations 1010 through 1111 are not used and have no meaning in BCD.**

- Consider decimal 185 and its corresponding value in BCD and binary:

$$\mathbf{185_{10} = (0001\ 1000\ 0101)_{BCD} = (10111001)_2}$$

BCD code

- It is important to realize that BCD numbers are decimal numbers and not binary numbers, although they use bits in their representation. The only difference between a decimal number and BCD is that decimals are written with the symbols 0, 1, 2, c, 9 and BCD numbers use the binary code 0000, 0001, 0010, c, 1001. The decimal value is exactly the same.

- Decimal 10 is represented in BCD with eight bits as 0001 0000 and decimal 15 as 0001 0101. The corresponding binary values are 1010 and 1111 and have only four bits.

BCD Addition

- Consider the addition of two decimal digits in BCD, together with a possible carry from a previous less significant pair of digits. If we add the BCD digits as if they were binary numbers, the binary sum should equal to or less than 1001 (without a carry), then the corresponding BCD digit is correct.
- However, when the binary sum is greater than or equal to 1010 = (10 in decimal), the result is an invalid BCD digit.
- The addition of 6 = (0110) to the binary sum converts it to the correct digit and also produces a carry as required. This is because a carry in the most significant bit position of the binary sum and a decimal carry differ by 16 - 10 = 6.
- i.e This causes a **carry** to the next digit and resets the current one to a valid BCD.

- Consider the following three BCD additions:

4 0100	4 0100	8 1000
+5 +0101	+8 +1000	+9 1001
9 1001	12 1100	17 10001
	+0110	+0110
	10010	0001 0111

- In each case, the two BCD digits are added as if they were two binary numbers. If the binary sum is greater than or equal to 1010, we add 0110 to obtain the correct BCD sum and a carry.

- The addition of two *n*-digit unsigned BCD numbers follows the same procedure.

  Consider the addition of 184 + 576 = 760 in BCD:

BCD	1	1		
	0001	1000	0100	184
	+0101	0111	0110	576 +
Binary sum	0111	10000	1010	
Add 6	_____	0110	0110	
BCD sum	0111	0110	0000	760

Advantages and drawbacks of BCD Codes

- It is very similar to decimal system.
- We need to remember binary equivalent of decimal numbers 0 to 9 only.
- The addition and subtraction of BCD have different rules.
- The BCD arithmetic is little more complicated.
- BCD needs more number of bits than binary to represent the decimal number. So BCD is less efficient than binary.

Alphanumeric codes

- A binary digit or bit can represent only two symbols as it has only two states '0' or '1'. But this is not enough for communication between two computers because there we need many more symbols for communication. These symbols are required to represent 26 alphabets with capital and small letters, numbers from 0 to 9, punctuation marks and other symbols.

- The alphanumeric codes are the codes that represent numbers and alphabetic characters. Mostly such codes also represent other characters such as symbol and various instructions necessary for conveying information. An alphanumeric code should at least represent 10 digits and 26 letters of alphabet i.e. total 36 items.

The following three alphanumeric codes are very commonly used for the data representation.

- American Standard Code for Information Interchange *ASCII*.
- Extended Binary Coded Decimal Interchange Code *EBCDIC*.
- Five bit Baudot Code.
- ASCII code is a 7-bit code whereas EBCDIC is an 8-bit code.

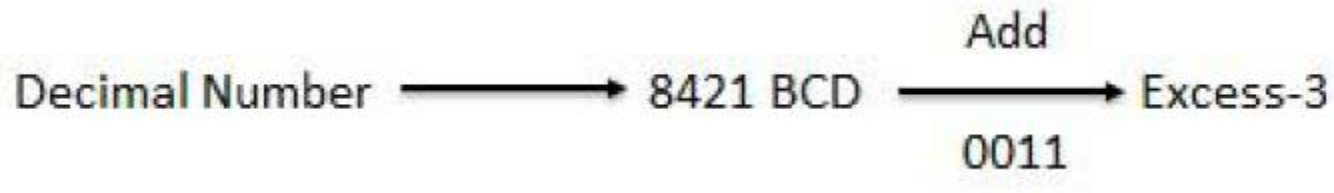
ASCII code is more commonly used worldwide while EBCDIC is used primarily in large IBM computers.

Non-Weighted Codes

- In this type of binary codes, the positional weights are not assigned. The examples of non-weighted codes are Excess-3 code and Gray code.

Excess-3 code

- The Excess-3 code is also called as XS-3 code. It is non-weighted code used to express decimal numbers. The Excess-3 code words are derived from the 8421 BCD code words adding 0011 to each code word in 8421. The excess-3 codes are obtained as follows:



Decimal	BCD 8 4 2 1	Excess-3 BCD + 0011
0	0 0 0 0	0 0 1 1
1	0 0 0 1	0 1 0 0
2	0 0 1 0	0 1 0 1
3	0 0 1 1	0 1 1 0
4	0 1 0 0	0 1 1 1
5	0 1 0 1	1 0 0 0
6	0 1 1 0	1 0 0 1
7	0 1 1 1	1 0 1 0
8	1 0 0 0	1 0 1 1
9	1 0 0 1	1 1 0 0

Gray Code (Reflected Binary Code RBC)

- It is a non-weighted code and not arithmetic codes. That means there are no specific weights assigned to the bit position. It has a very special feature that, only one bit will change each time the decimal number is incremented.

- As only one bit changes at a time, the gray code is called as a. unit distance code
- The gray code is a cyclic code and cannot be used for arithmetic operation.

Convert from Binary to Gray code

- To convert from a binary number (d_1, d_2 ,…., d_{n-1} , d_n) to gray code (g_1 g_2, ……, g_{n-1} , g_n), do the following:

1. Start with the right digit d_n. if the d_{n-1} is 1, replace d_n by (1- d_n); otherwise leave it unchanged.
2. Proceed to d_{n-1} and continue up to the first digit d_n which is kept the same since do is assumed to be 0.
3. The resulting (g_1 ,g_2, ……, g_{n-1} , g_n) is called the reflected gray code.
4. Example: convert the binary number 0111 to gray code.

   The answer is 0100

Convert from Gray code to Binary code

- To do the opposite:

Start again from the last digit (g_n), and compute

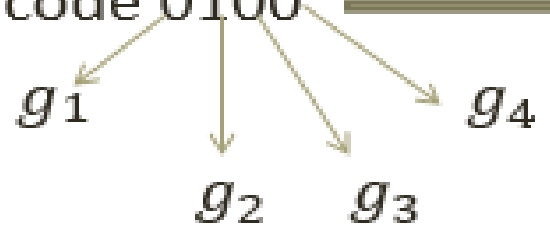
$$\sum n = \sum_{i=1}^{n-1} g_i \text{ (mode 2)}$$

- If $\sum n$ is 1 , replace g_n by (1- g_n), otherwise leave it unchanged.
- Next compute

$$\sum n - 1 = \sum_{i=1}^{n-2} g_i \text{ (mode 2)}$$

And so on, the resulting $(d_1, d_2, \ldots, d_{n-1}, d_n)$ is the binary number.

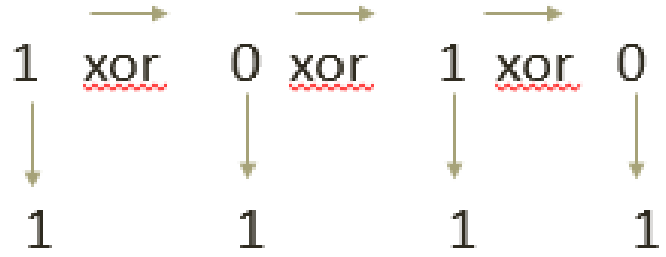
- The gray code 0100 ⟶ the binary code 0111



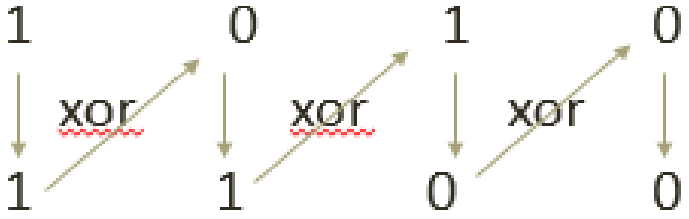
- (0 +1 + 0) mode 2 = 1 then g_4 = 0 is replaced by (1 - g_4) = (1 – 0) = 1.
- (0 + 1) mode 2 = 1, replace g_3 by (1 - g_3) = (1 – 0) = 1, then
- 0 mode 2 = 0 , ;leave g_2 unchanged.

Simpler Way:

- Convert Binary 1010 to Gray code:



- Convert Gray number code 1010 to binary code:



Decimal	BCD	Gray
0	0 0 0 0	0 0 0 0
1	0 0 0 1	0 0 0 1
2	0 0 1 0	0 0 1 1
3	0 0 1 1	0 0 1 0
4	0 1 0 0	0 1 1 0
5	0 1 0 1	0 1 1 1
6	0 1 1 0	0 1 0 1
7	0 1 1 1	0 1 0 0
8	1 0 0 0	1 1 0 0
9	1 0 0 1	1 1 0 1